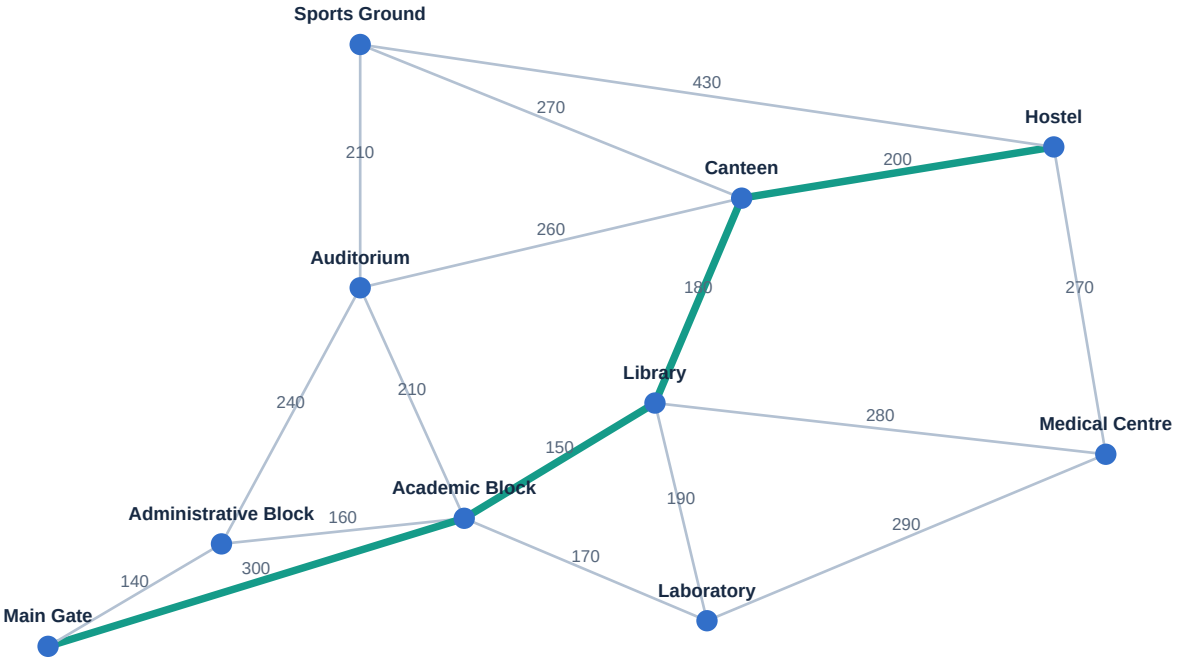
Smart Campus Navigator

Interactive Graph Algorithm Visualizer and Route Optimizer

Design and Analysis of Algorithms mini-project | Course code: 01AI0506



A fictional ten-location campus connects shortest-path learning with economical network planning. All six algorithms are implemented manually in Python and explained through inspectable events.

Student submission details

Student name	[Student name]
Enrollment number	[Enrollment number]
Guide name	[Guide name]
University	[University]

Editable fields above can be filled in a compatible PDF reader. All campus coordinates and distances are illustrative, not a real university survey.

Abstract and introduction

Abstract

Smart Campus Navigator models a fictional university as a weighted undirected graph and makes six fundamental algorithms visible. It addresses two distinct tasks: students seek short walking routes, while administrators seek a connected pathway network with minimum total length. BFS and DFS support exploration, Dijkstra finds single-source weighted shortest paths, Floyd-Warshall computes all-pairs distances, and Prim and Kruskal construct minimum spanning trees or forests.

The project uses Python 3.11 or newer, Streamlit for controls, and Plotly for an illustrative campus diagram. The algorithms rely on standard-library data structures and handwritten logic rather than graph-solving packages. Compact event histories expose the choices made during execution. A session-level editor permits controlled experiments with distances and connectivity.

On the original dataset, the shortest Main Gate-to-Hostel route is 830 metres. Both MST methods produce a total construction length of 1690 metres across nine selected edges. These outputs are computed from the actual dataset and checked by automated tests. Repeated timing measurements are recorded separately and interpreted as machine-dependent experiments, not as proofs of asymptotic complexity.

Introduction

A graph is a natural representation whenever places are linked by available movement. Straight-line proximity alone is insufficient: a student can only use listed walkable paths, and an indirect route may cost less than another available connection. The adjacency-list model records exactly those connections and their walking distances.

A visual interface helps bridge pseudocode and execution. A learner can inspect a Dijkstra relaxation, compare traversal orders, and see why a cycle edge is rejected during Kruskal. The same fixed network supports all examples, reducing the burden of learning a new dataset for each technique.

Project contribution

The work combines readable independent algorithm functions, deterministic graph editing, a local interactive dashboard, known-answer testing and beginner-oriented learning materials. The contribution is an educational integration of established algorithms; it does not claim a new routing algorithm or a real campus deployment.

Problem, objectives and syllabus

Problem statement

Given ten campus locations and seventeen bidirectional paths with finite nonnegative distances, compute useful routes and explain network structure. The application must answer a selected journey, analyze all location pairs, demonstrate reachable exploration, and construct a connected network using the least total pathway length when a spanning tree exists.

Objectives

• Represent one fictional campus consistently in code, diagrams and examples.

• Implement BFS, DFS, heap-based Dijkstra, Floyd-Warshall with next hops, heap-based Prim, and Kruskal with Disjoint Set Union.

• Explain individual decisions through compact step events and visible highlights.

• Support path changes and clearly distinguish unreachable routes and disconnected forests.

• Relate operation counts to Big O and verify outcomes with automated tests.

Syllabus relevance

The supplied brief identifies course 01AI0506, Unit 4 (Greedy Method and Graph Algorithms), Unit 1 support, and Practicals 8-11. No separate official syllabus document was included. This report therefore uses the requested mapping and does not assign invented official practical titles. Dijkstra, Prim and Kruskal demonstrate greedy choice; BFS and DFS demonstrate graph traversal; Floyd-Warshall demonstrates dynamic programming for all-pairs routes.

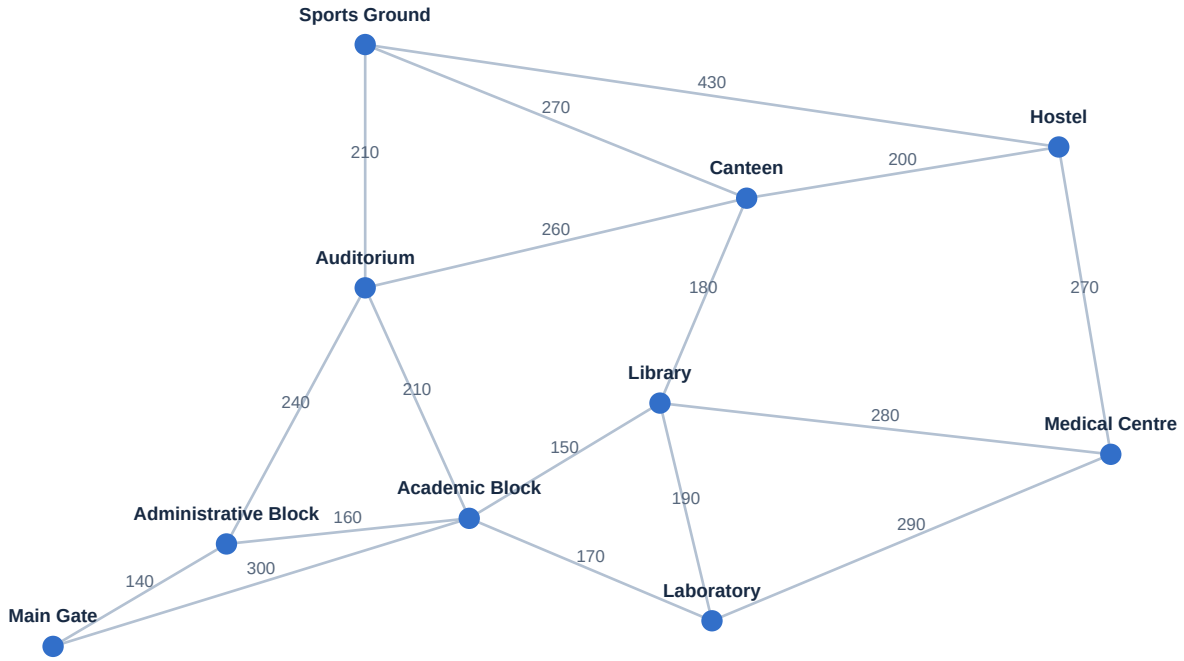
Unit 1 support appears in input-size definitions, time and space bounds, efficiency discussion, and measured performance. A viva explanation should distinguish theoretical analysis from observed runtime. Exact practical numbering should be reconciled with the department copy if it differs from the brief.

Existing approach and proposed solution

A static campus sketch shows paths but requires manual route addition. A spreadsheet can hold distances but does not reveal the search process. The proposed system links computations to a diagram and readable events, allowing a learner to change an edge and explain the resulting answer. It stays local and small enough for an examiner to inspect the complete logic.

Graph model and system architecture

Vertices: Main Gate, Administrative Block, Academic Block, Library, Laboratory, Auditorium, Canteen, Hostel, Sports Ground and Medical Centre. An edge stores one walking distance in metres and appears in both endpoint adjacency dictionaries. The dataset is simple: no self loops and at most one edge per unordered pair.




```
data/campus.json -> Graph validation / session edits
                          |
                 algorithms.py (six methods)
                          |
                 Result / AllPairs + events
                          |
                 app.py + visualization.py
                          |
                 Streamlit controls + Plotly
```

Coordinates are local illustrative positions in metres. Original path lengths are at least their endpoint straight-line separations; they may be longer because walking paths bend. Computations use edge weights, not coordinate geometry. User edits may deliberately be unrealistic to expose algorithm behavior.

The graph layer owns validation. The algorithm layer owns answers and history. The interface owns selection, session state and rendering. Tests call the same public functions used by the app. Report and slide generation import the original dataset and recorded measurements, avoiding a second hand-maintained source of sample answers.

Traversal: BFS and DFS

Breadth-First Search

BFS uses a FIFO queue. A vertex is marked when enqueued so multiple incoming edges do not queue it twice. Removing the oldest queued vertex explores the graph in layers. In an unweighted graph those layers correspond to minimum edge counts, not minimum weighted distance.

```
mark source; enqueue source
while queue is not empty:
  u = dequeue; output u
  for each neighbor v of u:
    if unseen: mark v; enqueue v
```

Triangle example A-B=2, B-C=1, A-C=5: from A the queue becomes [B,C], then each is removed. C is one hop away through the costly direct edge; this is why a weighted route finder uses Dijkstra.

Depth-First Search

DFS explores a branch until it has no unseen neighbor, then backtracks. This implementation uses a stack of suspended neighbor iterators to reproduce recursive DFS without a recursion-depth limit. Every reachable vertex is entered once, and every neighbor list is completely scanned.

```
mark and output source; push (source, neighbors)
while stack is not empty:
  inspect next neighbor of top frame
  if exhausted: pop frame
  elif unseen: mark, output, push its frame
```

On the same triangle DFS enters A, then B, then C, before popping exhausted frames. The visit order happens to match BFS on three vertices, but the exploration mechanism differs. On the campus DFS enters deep hostel and medical branches before returning to Sports Ground.

Both methods run in O(V+E) time with adjacency lists: process each vertex once and inspect each undirected edge twice. Queue/stack, seen set and returned order occupy O(V) space. A disconnected graph returns only the starting component; the UI reports the reached count.

Dijkstra: shortest walking routes

Dijkstra maintains the best known distance to each location, starting with zero at the source and infinity elsewhere. A binary min-heap chooses a smallest tentative vertex. Once settled, a vertex is final because nonnegative remaining edges cannot create a cheaper route through an unsettled vertex. Relaxation proposes improved distances through the settled location.

```
distance[source] = 0; other distances = infinity
push (0, source)
while heap is not empty:
  pop minimum (distance, u); skip if settled
  settle u
  for (v, weight) adjacent to u:
    if distance[u] + weight < distance[v]:
      update distance[v], parent[v]; push new entry
```

Worked campus relaxation

Main Gate proposes Academic Block=300 and Administrative Block=140. Administrative Block is settled first and proposes Auditorium=380. Its alternative Academic Block route is also 300, so the strict improvement test keeps the existing direct parent. Academic Block proposes Library=450 and Laboratory=470. Auditorium first proposes Canteen=640; Library later improves it to 630. Canteen gives Hostel=830.

Following parents backward from Hostel gives Hostel, Canteen, Library, Academic Block, Main Gate. Reversing yields the displayed route, with 300+150+180+200=830 metres. The route finder completes all reachable single-source distances; its ten settled nodes reflect that full computation.

Correctness and implementation assumptions

When the smallest tentative distance is settled, no route through an unsettled vertex can improve it: reaching that vertex already costs at least as much, and remaining edges are nonnegative. Relaxation preserves upper bounds until all reachable distances are final.

The graph rejects negative or nonfinite weights and permits zero-weight edges. Lazy heap entries avoid decrease-key code; stale settled entries are skipped. A source equal to its destination yields [source] at zero distance. An unreachable destination keeps infinity and returns an empty path.

For a simple graph the binary-heap implementation takes O((V+E) log V) time. The lazy heap may retain O(E) entries, giving O(V+E) auxiliary/result memory, excluding the graph. Parent reconstruction uses O(V) time and output space in the longest simple route.

Floyd-Warshall: every pair

Floyd-Warshall is useful when many source-destination questions are asked on a small unchanged graph. It starts with zero on the diagonal, edge weights for direct connections and infinity for missing connections. A next-hop table supports reconstruction instead of storing only distances.

```
initialize direct distances and zero diagonal
for each intermediate k:
  for each source i:
    for each destination j:
      if d[i,k] + d[k,j] < d[i,j]:
        d[i,j] = d[i,k] + d[k,j]
        next[i,j] = next[i,k]
```

Intermediate-node invariant

After intermediate k, d[i,j] is the shortest distance using only the intermediates processed so far. An optimal path either avoids k or splits into i-to-k and k-to-j subpaths. The minimum of these two cases preserves the invariant.

For A-B=2, B-C=1 and A-C=5, allowing B as an intermediate improves d[A,C] to 3. The update stores next[A,C]=B. Reconstruction begins at A, follows B, then reaches C. The same reasoning applies to every campus pair and the all-pairs table reports 830 metres from Main Gate to Hostel.

Selected computed distance entries

From	To	Shortest distance (m)
Main Gate	Library	450
Main Gate	Auditorium	380
Main Gate	Hostel	830
Library	Medical Centre	280
Hostel	Sports Ground	430

The outer intermediate loop and two endpoint loops each run V times, so time is $O(V^3)$. The two V-by-V tables use $O(V^2)$ space with history disabled. Compact update events may number $O(V^3)$, an additional tracing cost explicitly separated in the complexity discussion.

The general algorithm can handle negative directed edges without negative cycles, but this project uses the shared nonnegative undirected graph contract. An unreachable pair has no next hop. Selecting the same source and destination reconstructs a one-vertex zero-distance route.

Minimum network: Prim and Kruskal

An MST includes all vertices, has no cycle, and minimizes the sum of selected edge weights. With construction cost assumed proportional to length, it is a simple model of economical campus connectivity. It does not minimize all individual journeys. For example, Main Gate-to-Auditorium is 510 m in the MST but 380 m in the full graph.

```
for each component root not yet visited:
  push (0, root, no parent)
  while heap is not empty:
    pop cheapest connection; skip visited endpoint
    visit endpoint; accept edge if parent exists
    push its edges to unvisited vertices
```

```
make a separate set for every vertex
for edges in increasing (weight, endpoints) order:
  if endpoint roots differ:
    accept edge; union the two sets
  else: reject cycle edge
```

Prim grows a component by taking a cheapest crossing edge from a min-heap. Kruskal considers globally sorted edges; DSU determines whether they join separate components. Path compression and union by rank keep repeated component queries efficient. Both follow the cut property: a cheapest edge across a cut is safe for some MST.

Method	Original campus result	Selection rule
Prim	1690 m; 9 edges	Cheapest crossing edge
Kruskal	1690 m; 9 edges	Sorted edges; reject cycles

Prim takes O((V+E) log V) time and O(V+E) auxiliary/result space with its lazy edge heap. Kruskal takes O(V+E log E), usually written O(E log E) for connected graphs, and O(V+E) space. If input is disconnected, the code returns an optimal forest across components and explicitly says that a full spanning tree cannot be constructed.

Equal weights can permit different optimal MSTs, all with the same total. This campus happens to have a unique MST; a complete four-vertex equal-weight test graph exercises ties. Deterministic heap and sort keys keep live demonstrations reproducible.

Implementation and verification method

Implementation methodology

1. Define and validate one JSON dataset. Graph.set_edge updates both directions, rejects unknown endpoints, self loops and invalid distances, and sorts affected neighbor lists at edit time. Graph.remove_edge removes both directions. Reloading the JSON restores the original.

2. Implement algorithms in independent functions. Result contains traversal order, route, distances, tree edges, total, connectivity status and events as applicable. AllPairs stores distance and next-hop tables. trace=False suppresses history storage for fairer timing.

3. Build Streamlit sections around those public functions. Plotly highlights only supplied result/step edges. Session state retains path edits and step indices; a graph revision gives new step keys after edits. Previous, Next and Reset are explicit manual controls, avoiding fragile timer behavior during a viva.

4. Check behavior before documenting it. Automated tests use small known-answer graphs, validate exceptions and edge cases, compare Dijkstra against Floyd-Warshall, and enumerate tiny candidate trees as an independent MST oracle. Streamlit integration tests exercise all six pages and form callbacks.

Representative actual checks

Case	Expected and tested behavior
A-B=2, B-C=1, A-C=5, C-D=3, B-D=8	A-to-D = 6 via A,B,C,D; MST total = 6
Zero-weight triangle	A-to-C = 0; both MST totals = 0
Disconnected A-B=2, C-D=1, isolated E	A-to-E unreachable; forest total 3; connected=False
Same source and destination	Single-vertex path; distance 0
Campus shortcut edit	Main Gate-Hostel=10 makes route 10; restore returns 830
Long chain	Iterative DFS visits 1200 vertices without recursion

The committed test log records the executed unittest results. Finite tests support confidence but do not replace the invariants and cut-property arguments described earlier. Windows helpers are supplied for convenience; runtime checks in this workspace use Linux and Python 3.12.

Complexity and efficiency analysis

Big O is an upper bound on growth, suppressing constants and smaller terms. Here V means locations and E means undirected paths. An adjacency list stores each edge twice but still needs O(V+E) space. The table excludes graph storage and optional event history; it includes auxiliary structures and returned results.

Algorithm	Time	Space	Main reason
BFS	O(V + E)	O(V)	FIFO queue
DFS	O(V + E)	O(V)	Stack of iterator frames
Dijkstra	O((V + E) log V)	O(V + E)	Greedy; lazy binary min-heap
Floyd-Warshall	$O(V^3)$	$O(V^2)$	Dynamic programming over intermediates
Prim	O((V + E) log V)	O(V + E)	Greedy; lazy crossing-edge heap
Kruskal	O(V + E log E)	O(V + E)	Sort edges; Disjoint Set Union

Why these bounds fit this code

BFS and DFS process vertices once and scan each adjacency entry once. The queue, stack frames, visited set and order need O(V). Floyd-Warshall visits V intermediate choices times V source choices times V target choices; the distance and next-hop tables each have V^2 entries.

Dijkstra and Prim use lazy binary heaps, which may retain O(E) entries rather than only one entry per vertex. For this simple graph E is at most V(V-1)/2, so log E is O(log V). At most O(V+E) processing/push/pop work yields O((V+E) log V), with O(V+E) memory.

Kruskal initializes V DSU elements, sorts E edges in O(E log E), and performs O(E) union/find operations with inverse-Ackermann amortized cost. For nontrivial simple graphs sorting dominates that near-linear DSU work. The explicit O(V) term remains necessary when isolated vertices are present.

Trace and preparation overhead

Histories store compact event deltas rather than a full copy of every distance table. BFS/DFS/Dijkstra/Prim/Kruskal add O(V+E) events, while Floyd may add $O(V^3)$. Event strings and dictionary allocation increase measured constants. The graph sorts neighbor dictionaries during editing; that preparation is outside algorithm and timing bounds.

Comparisons should match objectives: BFS versus DFS for exploration and Prim versus Kruskal for network construction. Dijkstra computes one source; Floyd-Warshall computes every pair. A faster single-source timing does not establish that it is universally better for repeated all-pairs workloads.

Demonstration and measured results

Computed campus outputs

Shortest Main Gate-to-Hostel route: Main Gate - Academic Block - Library - Canteen - Hostel. Distance: 830 m. MST total: 1690 m with nine edges. The same source/target distance appears in the Floyd-Warshall table. These values are generated from the current original JSON dataset and asserted in tests.

Prim accepts (metres): Gate-Administrative 140; Administrative-Academic 160; Academic-Library 150; Academic-Laboratory 170; Library-Canteen 180; Canteen-Hostel 200; Academic-Auditorium 210; Auditorium-Sports 210; Hostel-Medical 270. The displayed application uses the full location names.

Experimental timings: actual recorded run

Python 3.12.14 / Linux-6.18.55-x86_64-with-glibc2.41. Each workload receives one warm-up followed by seven perf_counter measurements. The table reports median milliseconds. Graphs are a guaranteed connected chain plus seeded extra edges with probability 0.12. Input generation and explanatory history are excluded.

Algorithm	V=12, E=17	V=24, E=51	V=48, E=165
BFS	0.0077	0.0143	0.0326
DFS	0.0088	0.0186	0.0430
Dijkstra	0.0170	0.0406	0.1062
Floyd-Warshall	0.2991	2.0585	14.7046
Prim	0.0138	0.0318	0.0883
Kruskal	0.0208	0.0592	0.1821

Small timings are strongly influenced by interpreter overhead and machine load. As graph size increases, both vertex count and edge density change. These observations illustrate this run; they neither prove Big O nor provide an objective-neutral ranking. Re-run campus.benchmark to obtain measurements for another machine.

Live demonstration sequence

Start at Dashboard, show the 830-m route, compare BFS/DFS, inspect both 1690-m MSTs, and check the all-pairs matrix. Add a temporary shortcut to demonstrate recomputation, then restore the original graph. docs/DEMO_SCRIPT.md gives a timed five-minute sequence with exact control labels.

Conclusion, scope and references

Advantages and limitations

The project keeps established algorithms visible in short readable functions, reuses one dataset and explains results with deterministic steps. It requires no account, key, database or external map service. After dependencies are installed, local execution supports an offline classroom demonstration.

The campus is fictional, distances are static and symmetric, and construction cost is assumed proportional to length. The model omits accessibility, crowding, slopes, permissions and turn restrictions. A minimum tree also has no redundant links. Session changes are temporary. Floyd-Warshall and extensive event histories suit small teaching graphs rather than city-scale road networks.

Future scope

Useful extensions would collect validated real campus data with permission, mark accessibility constraints, add travel-time scenarios and persist reviewed graph versions. Each would need a clear data contract and new tests. The present submission deliberately avoids unrelated algorithms and external-service dependencies.

Conclusion

Smart Campus Navigator turns six syllabus algorithms into an inspectable local demonstration. The tested campus route is 830 m, while the optimal network spans all ten locations with 1690 m of pathway. The distinction between those objectives, together with invariants, complexity analysis and edge-case handling, provides the main academic learning outcome.

References

[1] Cormen, Leiserson, Rivest and Stein. Introduction to Algorithms, 4th edition. MIT Press, 2022. Sections on traversal, shortest paths and minimum spanning trees.

[2] Python documentation: collections.deque, heapq, time.perf_counter and unittest. https://docs.python.org/3/library/

[3] Streamlit documentation. https://docs.streamlit.io/

[4] Plotly Python documentation. https://plotly.com/python/

[5] ReportLab User Guide. https://docs.reportlab.com/reportlab/userguide/

[6] python-pptx documentation. https://python-pptx.readthedocs.io/

[7] User-provided project brief: course 01AI0506; Unit 4 focus, Unit 1 support and Practical 8-11 mapping. A separate official syllabus was not attached.